

1

main.py



Share

Run

```

1 from collections import deque
2
3 def bfs(graph, start):
4     visited = set()
5     queue = deque([start])
6     visited.add(start)
7     order = []
8
9     while queue:
10         vertex = queue.popleft()
11         order.append(vertex)
12
13         for neighbor in graph[vertex]:
14             if neighbor not in visited:
15                 visited.add(neighbor)
16                 queue.append(neighbor)
17
18     return order
19
20
21 def dfs(graph, start, visited=None, order=None):
22     if visited is None:
23         visited = set()
24
25     if order is None:
26         order = []

```

2

main.py



Share

Run

```

23     visited = set()
24
25     if order is None:
26         order = []
27
28     visited.add(start)
29     order.append(start)
30
31     for neighbor in graph[start]:
32         if neighbor not in visited:
33             dfs(graph, neighbor, visited, order)
34
35     return order
36
37
38 graph = {
39     'A': ['B', 'C'],
40     'B': ['A', 'D', 'E'],
41     'C': ['A', 'F'],
42     'D': ['B'],
43     'E': ['B', 'F'],
44     'F': ['C', 'E']
45 }
46
47 print("BFS Traversal starting from 'A':", bfs(graph, 'A'))
48 print("DFS Traversal starting from 'A':", dfs(graph, 'A'))

```

3

Output

```

BFS Traversal starting from 'A': ['A', 'B', 'C', 'D', 'E', 'F']
DFS Traversal starting from 'A': ['A', 'B', 'D', 'E', 'F', 'C']

```

```

=== Code Execution Successful ===

```